The Machine Learning in FGM

The Guildlines

The Combination of Machine Learning and Openfoam

Student Information

Name **Zhe Wang**
Stud. ID **519021910104**

March, 2022

UM-SJTU Joint Institute

Contents

1 Introduction

This guildline is aimed at guilding you how to use the idea of Machine Learning in the calculation process of Openfoam. In this article, I will show you how to replace the look-up table procedure in flamelet-type models by ANN (Artificial Neural Network)

2 The construction of ANN

This section can be divided into the following three parts:

- **The construction of the ANN model.**
- **The construction of the training data.**
- **The errors that you may meet.**

2.1 The construction of the ANN model:

2.1.1 Install Python3 environment

The ANN model in this project is based on the Pytorch that means the code is written in Python language. So configuring the Python environment should be your first step. Here, I strongly recommand you to install python3 as it supports much more libraries that the previous version. For detailed instructions of insatlling python3, I think google it may be a better choice than finding answers here (Of course, baidu also has plenty of detailed guildlines).

2.1.2 The code of the ANN

For this project, we only need *"2In3Out.py"* and *"netTest.py"*. The netTest.py contains the basic construction of the neural network including the number of layers and the active functions. (We use relu function as the active function in this project.) The "2In3Out.py" contains all the other aspects of the model. In this project, the modifications will be focus on the 2In3Out.py.

The code in this file can be divided into seven parts:

- **The preparing section:**

  In this section, we set the number of neurons of each layer, the dimension of the input data and the output (or you can say the prediction). Also the training times and learning rate can be modified here.

- **The data loading section:**

  In this section, the training data (the input samples and the corresponding tags) of the ANN model is set.

```
67  nZ=1001
68  nPV=501
69  Ztarget = np.linspace(0,1,nZ)
70  Ctarget = np.square(np.linspace(0,1,nPV))
71
72
73  x=Ztarget
74  y=Ctarget
75
76  inputs_np = np.zeros((len(x)*len(y),INPUT_FEATURE_DIM ))
```

Figure 2: Loading Z and C (inputs)

```
20  # 训练次数
21  TRAIN_TIMES = 30000#2040000#68*30000
22  PATIENCE = 100
23  PRIDICTION_TOLERANCE = 0.01#1%TOLERANCE of the real data
24  # 输入输出的数据维度，这里都是1维
25  INPUT_FEATURE_DIM = 2
26  OUTPUT_FEATURE_DIM = len(fieldNames)
27  # 隐含层中神经元的个数
28  HIDDEN_1_DIM = 8
29  HIDDEN_2_DIM = 16
30  HIDDEN_3_DIM = 32
31  HIDDEN_4_DIM = 16
32  HIDDEN_5_DIM = 8
33
34  LEARNING_RATE = 0.001
35  print(OUTPUT_FEATURE_DIM)
```

Figure 1: The preparing stage, code 21-35

```
95  outputs_np = np.zeros((len(x)*len(y),OUTPUT_FEATURE_DIM))
96  index = -1
97
98  #数据标签的预处理
99  for Z in range(len(x)):
100     for C in range(len(y)):
101         index += 1
102         inputs_np[index] = np.array([x[Z],y[C]])
103         for i, iFGM in enumerate(NFGM):
104             outputs_np[index][i] = iFGM.at[Z,C]#这是建立了个二维表，x和y轴分别为C和Z
105
106 x_data = torch.from_numpy(inputs_np).float()
107 y_data = torch.from_numpy(outputs_np).float()
108
109
110 #下面是对标签的归一化处理
111 y_data_real = y_data
112 Min = y_data_real.min(0).values
113 Max = y_data_real.max(0).values
```

Figure 3: Loading the tags (outputs)

The codes in line 110 to 113 are for normalization of the tags.

- **Choose the ANN model**

  This part is quite simple as it just calls the netTest.py to creat a basic structure of an ANN.

```
95  # ============================ step 2/6 选择模型 ============================
96
97  # 建立网络
98  net = netTest.Batch_Net_5_2(INPUT_FEATURE_DIM, HIDDEN_1_DIM, HIDDEN_2_DIM, HIDDEN_3_DIM, HIDDEN_4_DIM, HIDDEN_5_DIM, OUTPUT_FEATURE_DIM)
99  # init_params(net)
00  print(net)
```

Figure 4: Construct the ANN

- **Choose the optimizer**

  For this project, the Adam optimize algorithm is chosen as it can ensure the rate of finding the extreme point as well as prevent random walk in the optimal direction of each training.

```
# ============================== step 3/6 选择优化器    ==========================

optimizer = torch.optim.Adam(net.parameters(), lr=LEARNING_RATE)
```

Figure 5: Optimizer

- **Choose the Loss Function (the Target Function)**

  The MSE loss function is chosen for this project. If you want to change a loss function, a L2 type loss function should by a much better choice than L1 type loss function.

```
# ============================== step 4/6 选择损失函数 ==========================
# 定义一个误差计算方法
# Mean Square Error (MSE), MSELoss(), L2 loss
# Mean Absolute Error (MAE), MAELoss(), L1 Loss
loss_func = torch.nn.MSELoss()
INITIAL_LOSS = loss_func(0*y_data, y_data)
LOSS_TOLERANCE = loss_func(0*y_data, PRIDICTION_TOLERANCE*y_data).data.numpy()
print('LOSS_TOLERANCE',LOSS_TOLERANCE)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, factor=0.2, threshold=PATIENCE/TRAIN_TIMES*LOSS_TOLERANCE, patience=PATIENCE)
```

Figure 6: The Loss Function

- **The training process**

```
# ============================== step 5/6 模型训练 ==============================
for i in range(TRAIN_TIMES):
    # 输入数据进行预测
    prediction = net(x_data)
    #prediction_real = prediction*(Max-Min)+Min
    # 计算预测值与真值误差，注意参数顺序问题
    # 第一个参数为预测值，第二个为真值
    loss = loss_func(prediction, y_data)#*100/firstLoss
    # Change the learning rate
    #scheduler.step()
    scheduler.step(loss)
    # 开始优化步骤
    # 每次开始优化前将梯度置为0
    optimizer.zero_grad()
    # 误差反向传播
    loss.backward()
    optimizer.step()
    # 按照最小loss优化参数
    # 可视化训练结果
    LOSS_SQRT = np.sqrt(loss.data.numpy()/INITIAL_LOSS.data.numpy())
    print("Iteration : "+'%05d'%i, "\tLearningRate : {:.3e}\tLoss: {:.4e}\tRelativeError:{:.5e}".format(_optimizer.param_groups[0]['lr'], loss.data.numpy(), LOSS_SQRT_))
    # if LOSS_SQRT < PRIDICTION_TOLERANCE:
        # break    # Lower than tollance break here
```

Figure 7: The code for training process

- **Save the model you have trained**

```
141  # ============================ step 6/6 保存模型 ============================
142  traced_script_module = torch.jit.trace(net, torch.rand(INPUT_FEATURE_DIM))
143  traced_script_module.save(graphsPATH+'combinedNet.pt')
144  torch.save(net.state_dict(), graphsPATH+'combinedNet.pkl')
```

Figure 8: Save Model

The graphsPATH here is just the directory where you want to store the model in. If you want, you can delete it.

2.2 The construction of the training data:

First, we should decide which species that we want to predict and add them to the "fieldNames" (code line 16).

```
fieldNames = ['RHO', 'DIFF', 'VISC', 'CP', 'T', 'CO2', 'O2', 'CO', 'CH4', 'H2O', 'H2', 'OH', 'H', 'O', 'HO2', 'H2O2', 'C', 'CH', 'CH2',
```

Figure 9: The fieldNames

Then, we need to constract our data base. The naming rules and location should follow the code line 41 or you can modify it, of course.

```
speciesI = pd.read_csv('01.orgData/Y-'+iname+'.txt', header=None)
```

Figure 10: code line 41

The Input data Z and C:
For the input data reading, I replace the original code

```
67  nZ=1001
68  nPV=501
69  Ztarget = np.linspace(0,1,nZ)
70  Ctarget = np.square(np.linspace(0,1,nPV))
```

with code

```
Ztarget = []
Ctarget = []
fhandZ = open('theZ.txt')
fhandC = open('theC.txt')
for line in fhandZ:
    Ztarget.append(float(line))
for line in fhandC:
    Ctarget.append(float(line))
```

As a result, we can custom our own data of Z and C and store them in *TheZ.txt* and *TheC.txt*, the data's typesetting in the two file should be like in the following form:

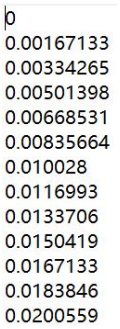

Figure 11: The typesetting of Z and C data

each line represents a data.
The output data (The tag):
Take "RHO" data as an example.

1. Make a folder and name it as *01.orgData*. The Linux command should be *mkdir 01.orgData*.
2. Create a txt file and name it as *Y-RHO.txt*. The Linux command should be *touch Y-RHO.txt*
3. The data in the txt file should be considered as a determinant. The index of Z can be viewed as the index of rows while the index of C can be viewed as the index of columns. For example, (Z[0], C[1]) should be the data at the first row with the second column. The data corresponding to different C values should be separated by comma, and the data corresponding to different Z values should be in different lines as shown below:

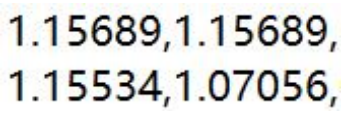

Figure 12: The sample of data

In this case, (Z[0],C[0]) is 1.15689, while (Z[1],C[1]) is 1.07056.

2.3 The errors that you may meet

- **Error:** No module named 'netTest'

  Solution: The netTest.py should be put in the same folder as 2In3Out.py.

- **Error:** The Loss and relative Error are nan.

  Solution: This problem is caused by *Vanishing gradient*. Please make sure that your training tags are not constant. For example, if the data in Y-RHO.txt is a constant, then it's very likely to meet Vanishing gradient. Just delete the corresponding species in the fieldNames can be help.

3 How to use ANN

This section just wants to tell you how to run the trained ANN model in Python. The application in OpenFoam will be showed in next section. So, if you are not interest in it, you can just skip to the next section.

As mentioned in the construction of ANN, we have saved two things in the "Save section" (Figure 9). One is "combinedNet.pt" and the other one is "combinedNet.pkl". The only difference between these two files is the way of how to load.

For the file that end with ".pt", it contains the whole network. You can load it by one line of code:

$$\text{model} = \text{torch.load("combinedNet.pt")}$$

For the file that end with ".pkl", the way of loading is more complex than the previous one. But it requires much smaller disc space as it only contains parameter of each layer. That means you should construct a basic ANN first and then load the file to complete the whole ANN. The detailed codes of using the pkl file are shown below. (Actually, whether containing the whole net or not is not decided by the suffix but depends on "net.state_dict()" in torch.save() function.)

```
import torch
import numpy as np
import pandas as pd
from matplotlib import pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import torch.nn.functional as F
import netTest
fieldNames = ['RHO', 'DIFF', 'VISC', 'CP', 'T', 'CO2', 'O2', 'CO', 'CH4', 'H2O', 'H2', 'OH', 'H', 'O', 'HO2', 'H2O2', 'C', 'CH', 'CH2', 'CH2(S)', 'CH3', 'H
# ,'C7H14OOH','CH3CHO','H2O','NC7KET','C2H3','C3H5-A','C7H14O','CH3COCH2','H2','O2','C2H4','C3H5O','C7H15-1','CH3CO','HCCO','OH','C2H5CHO','C3H6','C7H15O2'
# 输入输出的数据维度，这里都是1维
INPUT_FEATURE_DIM = 2
OUTPUT_FEATURE_DIM = len(fieldNames)
# 隐含层中神经元的个数
HIDDEN_1_DIM = 8
HIDDEN_2_DIM = 16
HIDDEN_3_DIM = 32
HIDDEN_4_DIM = 16
HIDDEN_5_DIM = 8
LEARNING_RATE = 0.001
net = netTest.Batch_Net_5_2(INPUT_FEATURE_DIM, HIDDEN_1_DIM, HIDDEN_2_DIM, HIDDEN_3_DIM, HIDDEN_4_DIM, HIDDEN_5_DIM, OUTPUT_FEATURE_DIM)
model = net.load_state_dict(torch.load('network.pkl'))
Z = input("Print the Z value: ")
C = input("Print the C value: ")
try:
    Z = float(Z)
    C = float(C)
    test_in = np.array([Z, C])
except:
    print("Not available Z or C")
    quit()
x_data = torch.from_numpy(test_in).float()
prediction = net(x_data)
print(prediction)
print(prediction[0])
exit()
```

Figure 13: How to use pkl model

Attention:

The result may not be the same as the original tags we have. That because the tags we have used to train the model are the normalizd tags instead of the original ones. According to the way of normalization, there is always a way to restore the tags.

4 The combination with OpenFoam

According to my senior's suggestion, I decide to modify an existing Solver in OpenFoam instead of creating a new one.

1. **Download the libtorch for C++**

Because the OpenFoam is written by C++, thus we need to download the Pytorch for C++.

Go to the website "https://pytorch.org/get-started/locally/", and choose the options as followed:

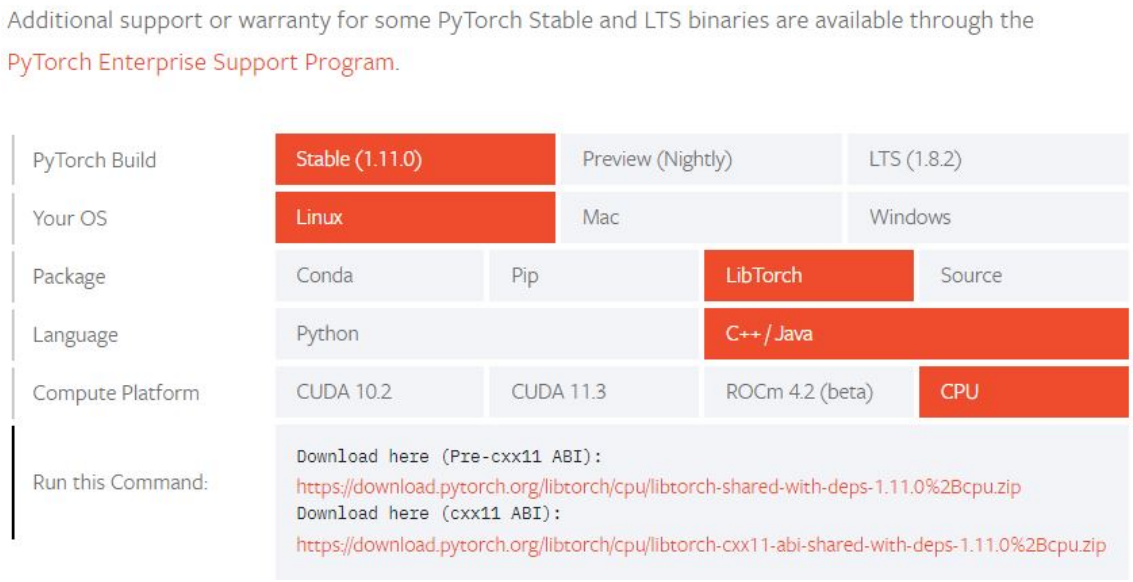

Figure 14: Download the pytorch for linux

Then we can get two download links, **copy the second links**, and run the command line

wget https://download.pytorch.org/libtorch/cpu/libtorch-cxx11-abi-shared-with-deps-1.11.0%2Bcpu.zip

to download the zip file. After that you can just decompression the zip file into a directory, such as ~/pyTorch.

(Attention: The following section is based on the conditions that I have decompressed the zip file into ~/pyTorch.)

2. **The code for using the model**

   Here, I will take the existed solver "RFPVFoamB" as an example to illustrate.

   In the solver RFPVFoamB, there is a function library calls tableFPV and I put all the code in printthedata function in tableFPV.C file. The code is shown below:

```
void tableFPV::printthedata(){
    //load the net of ML
    torch::jit::script::Module torchModel_ = torch::jit::load("network.pt");
    float A = 0.1;
    float B = 0.2;
    torch::Tensor inputs(torch::zeros({2}));
    inputs[0] = A;
    inputs[1] = B;
    std::vector<torch::jit::IValue> INPUTS{inputs};
    torch::Tensor output = torchModel_.forward(INPUTS).toTensor();
    float ans = output.accessor<float,1>()[0];
```

Figure 15: How to use ANN model in Linux

The float A and float B corresponding to the Z and C respectively.

Then upload the "network.pt" that you have trained into the root directory of RFPVFoamB. In this case, it should be the $gri30_tribrach_FPV_43$.

Also, please don't forget to call the function in the main function of the solver that you want to modify. In my case, the RFPVFoamB, of course.

```
(base) zhe_wang@dezhizhou:~/Openfoam/OpenFOAM-6/applications/solvers/combustion/RFPVFoamB$ ll
total 892
drwxrwxr-x  4 zhe_wang zhe_wang   4096 Mar 17 21:34  ./
drwxrwxr-x 20 zhe_wang zhe_wang   4096 Mar  3 12:11  ../
-rwxrwxr-x  1 zhe_wang zhe_wang     37 Mar  3 12:14  allWmake*
-rwxrwxr-x  1 zhe_wang zhe_wang    132 Mar  3 12:11  createClouds.H*
-rwxrwxr-x  1 zhe_wang zhe_wang    203 Mar  3 12:11  createFieldRefs.H*
-rwxrwxr-x  1 zhe_wang zhe_wang   5476 Mar  3 13:14  createFields.H*
-rwxrwxr-x  1 zhe_wang zhe_wang   6446 Mar  3 12:11  lookupTable.H*
drwxrwxr-x  2 zhe_wang zhe_wang   4096 Mar  3 12:11  Make/
-rw-rw-r--  1 zhe_wang zhe_wang  31360 Mar 17 19:53  network.pt
-rwxrwxr-x  1 zhe_wang zhe_wang   2398 Mar  3 12:11  pEqn.H*
-rwxrwxr-x  1 zhe_wang zhe_wang   3611 Mar  3 12:11  RFPVFoamB.C*
-rwxrwxr-x  1 zhe_wang zhe_wang   1695 Mar  3 12:11  rhoEqn.H*
drwxrwxr-x  4 zhe_wang zhe_wang   4096 Mar 17 23:40  tableFPV/
-rwxrwxr-x  1 zhe_wang zhe_wang    447 Mar  3 12:11  UEqn.H*
-rw-rw-r--  1 zhe_wang zhe_wang    249 Mar  3 12:11  updation_history
-rwxrwxr-x  1 zhe_wang zhe_wang 802297 Mar  3 12:11 'WeChat Image_20210514090856.jpg'*
-rwxrwxr-x  1 zhe_wang zhe_wang    624 Mar  3 12:11  YcEqn.H*
-rwxrwxr-x  1 zhe_wang zhe_wang   1123 Mar  3 12:11  YEqn.H*
-rwxrwxr-x  1 zhe_wang zhe_wang   3217 Mar  3 12:11  ZEqn.H*
```

3. **Prepare the working environment**

   - Activate the cantera, by using the command *"conda activate cantera"*, and the terminal will be changed like this:

     ```
     (base) zhe_wang@dezhizhou:~$ conda activate cantera
     (cantera) zhe_wang@dezhizhou:~$
     ```

   - Open the Make/options of tableFPV and add the following code into the EXE_INC

     ```
     1  EXE_INC = \
     2      -std=c++14
     3      -L$(HOME)/pytorch/libtorch/lib/libc10.so \
     4      -I$(LIB_SRC)/pytorch/libtorch/include/torch/
       csrc/api/include
     5
     ```

     the symbol "\" is used for telling the compiler that this is the end of a line. If your code is the last line of the section, the \ is unnecessary.

     Then add the following code into the EXE_LIB

     ```
     1  EXE_LIBS = \
     2      -rdynamic \
     3      -wl,-rpath, $(HOME)/pytorch/libtorch/lib
       $(HOME)/pytorch/libtorch/lib/libtorch.so   $(HOME)/
       pytorch/libtorch/lib/libc10.so \
     4      -wl,--as-needed $(HOME)/pytorch/libtorch/
       lib/libc10.so \
     5      -lpthread
     6
     ```

     In line 3 you can see there is a space between libtorch.so and $(HOME), actually it's a **code indent.**

     if **there is EXE_INC or EXE_LIBS in the file**, the first line "EXE_INC = \" or "EXE_LIBS = \" shouldn't be added. Otherwise, you can just write one in the file based on the format I have shown above.

- Open the Make/options of the RFPVFoamB and add the following code into EXE_LIBS

```
            EXE_INC = \
                -rdynamic \
                -L$(HOME)/pytorch/libtorch/lib $(HOME)/
    pytorch/libtorch/lib/libtorch.so \
                -L$(HOME)/pytorch/libtorch/lib/libc10.so
    \
                -lpthread

```

- Open the bashrc in the etc directory of the OpenFoam that you have installed and add the following code in the end.

```
            export LD_LIBRARY_PATH=$HOME/pytorch/libtorch
    /lib:$LD_LIBRARY_PATH

```

4. **Compile and Run**

- Go to the etc directory and source the bashrc that you have just modified.
- Complie the solver that you have modified.
- Run the new solver. The result you get from the ANN will be the normalized one.